Lo que queda por hacer

LudusERP · plan de los dos últimos sprints, por orden de prioridad

Fecha: 9 de noviembre de 2026 **Código:** master en c8cb7a5, más `tienda-cliente` sin mergear

Apartados pendientes: 15

> **Cómo leerlo**
>
> Los apartados van en el orden en que conviene atacarlos: los de arriba desbloquean a los de abajo. Cada uno lleva una etiqueta de necesidad — IMPRESCINDIBLE no se puede entregar sin ello, IMPORTANTE se nota si falta, SI SOBRA TIEMPO es deuda que no bloquea la entrega. Lo que ya está terminado no aparece en esta lista: esto es solo lo que falta.

> **Ramas**
>
> Una rama corta por apartado, salida de master y mergeada en cuanto el apartado funcione, igual que se ha hecho con `apartado-de-ventas` o `tienda-cliente`. Así dos personas no se pisan y cada merge deja master arrancable.

> **El camino crítico**
>
> Es la cadena **6** → **7** → **8**: lleva de un colaborador que sube su juego a un cliente que lo compra, recibe su clave y se lo descarga. Es lo que el tercer proyecto necesita que exista de verdad, y la tienda que lo enseña ya está escrita, esperando en su rama.

SPRINT 1 · DEJAR EL TERRENO FIRME Y ABRIR EL CIRCUITO

1 Punto de control: recrear la base, arrancar y mergear la tienda

IMPRESCINDIBLE

> Hacer `docker compose down -v`, arrancar, comprobar que las nueve migraciones se aplican y que Hibernate acepta el esquema, y mergear `tienda-cliente` en cuanto se vea funcionando.

V2 se ha editado a mano varias veces, así que la base que haya en los equipos ya no cuadra con la huella que guarda Flyway. Y hay una rama entera sin arrancar: el compilador no valida ni el esquema, ni las consultas, ni las plantillas. Eso solo sale al arrancar. Es media hora y es la red de seguridad de todo lo demás.

Lo que se prueba por primera vez en ejecución: las imágenes de juego, las fotos de perfil, la V9, los diez videojuegos que siembra `DatosIniciales` y, sobre todo, **la tienda entera**, que son dos `@Query` con `coalesce` y parámetros de tres estados más una plantilla de catálogo que Thymeleaf no valida hasta que la sirve. De paso: que un colaborador entra y cae en la tienda, que un gestor de inventario abre Ventas, que un cliente que escriba `/ventas/pedidos` se come un 403 y que cerrar sesión cierra de verdad.

Rama recomendada: `punto-control-esquema`

> **Primero esto.** Cualquier apartado de abajo que se empiece sin este paso se construye encima de algo que no sabemos si arranca.

2 Cerrar las decisiones que bloquean código

IMPRESCINDIBLE

> Sentaros veinte minutos, cerrar las cuatro decisiones abiertas que impiden escribir el apartado 6 y dejarlas escritas en la memoria.

Son cuatro y las cuatro son baratas de decidir y caras de improvisar a mitad de código: dónde se guarda el ZIP de los juegos (carpeta del servidor o almacenamiento aparte; si el hosting borra el disco en cada despliegue, la carpeta no vale y los juegos desaparecen), si un juego de colaborador nace activo o lo tiene que activar alguien, si el colaborador pone precio o todos sus juegos son gratis, y si en la tienda conviven los juegos de distribuidor con los de colaborador.

Rama recomendada: `decisiones-pendientes` · solo toca `docs/`

> La primera bloquea el apartado 6 entero. La cuarta ya tiene respuesta provisional escrita en código: el servicio de la tienda pasa `soloColaborador = null`, o sea que conviven; si se decide lo contrario es cambiar ese `null` por `false`.

3 Arreglar las dos pantallas que hoy están rotas

IMPRESCINDIBLE

> Un error de plantilla que tumba el detalle de pedido y un formulario que se traga los cambios sin avisar. Las dos son de pantallas ya entregadas.

El detalle de pedido revienta. `ventas/detallePedido.html` pinta `l.videojuego.estudio`, y ese campo ya no existe: la decisión 4.15 lo sustituyó por la clave ajena al proveedor. Cualquier `/ventas/pedido/{id}` con líneas da un error de expresión y un 500. Es la única referencia a ese campo que queda en todo el código, y se arregla en una línea.

Mis datos pierde lo que escribes. El formulario de `clients/configuracionUsuario.html` no tiene `th:action` ni `method`, y `ControllerHome` solo tiene el GET: «Guardar» recarga la página y los cambios se van sin un aviso. La foto sí se guarda, eso ya funciona, y el `UsuarioService` es el sitio donde colgar el guardado nuevo. Falta decidir qué campos puede editar quien no es cliente.

Rama recomendada: `arreglos-rotos`

> Van en el sprint 1 y no con los arreglos cosméticos del 14 porque no son detalles: una tumba la pantalla y la otra pierde datos del usuario delante de él.

4 Datos de prueba de pedidos y facturas

IMPORTANTE

> Sembrar unos cuantos pedidos pagados con sus líneas y sus facturas, para que el módulo de ventas se pueda ver y probar.

`DatosIniciales` siembra departamentos, empleados, clientes, proveedores, colaboradores y diez videojuegos, pero **ni un pedido**. El resultado es que las tres pantallas de ventas que se entregaron el 5 de noviembre (listado, detalle y facturas con su PDF) se ven siempre vacías, y el fallo del apartado 3 ha podido vivir ahí sin que nadie lo notara justamente por eso. Con media docena de pedidos repartidos entre los cuatro clientes, en los cuatro estados, y dos o tres facturas, se prueba el listado, los filtros, los contadores, el detalle y la generación del PDF.

Rama recomendada: `datos-prueba-ventas`

> Va en `DatosIniciales` con el resto, no en una migración: los datos de prueba no se versionan en Flyway. La versión anterior de este plan mandaba cargar un `docs/datosPruebaPedidos.sql` que **no existe en el repositorio**.

5 Un test que arranque el contexto

IMPORTANTE

> Una sola clase de prueba que levante la aplicación contra la base y falle si algo no encaja.

No hay ni un test y `src/test` ni siquiera existe, mientras el pom declara ocho starters de prueba. Un test de carga de contexto habría pillado el choque de entidades del 29 de octubre y el de la columna `activo` del 4 de noviembre, y pillaría hoy mismo cualquier error en las consultas nuevas de la tienda, que es exactamente lo que no sabemos si funciona. Es una clase de quince líneas que convierte «hay que acordarse de arrancar» en «el build falla».

Rama recomendada: `test-contexto`

> Está en el sprint 1 a propósito, aunque el resto de la deuda de pruebas se quede para el final: este test es lo que protege los cinco apartados que vienen después.

6 El colaborador sube sus juegos

IMPRESCINDIBLE

> Crear la pantalla donde un colaborador da de alta su juego con sus imágenes y su ZIP, y queda publicado sin que nadie lo revise. Y darle, por fin, una forma de cambiar su contraseña.

Es la razón de ser del actor COLABORADOR y hoy no existe ni una línea: no hay paquete `templates/colaborador/`, ni servicio, ni controlador, y `VersionJuego` es una entidad que no usa nadie. Hay que crear `JuegoColaboradorService` y su impl, `ControllerJuegoColaborador`, `VersionJuegoDao`, las plantillas y su CSS, y tocar `SecurityConfig` y el menú lateral. La mitad del trabajo se reaprovecha: `AlmacenArchivos` ya guarda y borra con las rutas validadas, y el CRUD de imágenes de videojuego sirve de molde. Lo nuevo de verdad es el ZIP y que el juego se guarde con `colaborador_id` y sin proveedor.

La contraseña. Hoy no hay ninguna vía para cambiar la de un colaborador: ni el admin ni él mismo. El servicio ya lo soporta, pero el formulario de edición no ofrece el campo, y la pantalla de «mi cuenta» del colaborador no existe. Son seis líneas en `admin/formularioEditarCol.html`, el mismo campo que ya tiene el de empleado, más la pantalla propia cuando se hagan las suyas.

Rama recomendada: `juegos-colaborador`

> Depende del apartado 2: hasta decidir dónde vive el ZIP no se puede escribir la parte de la entrega del ejecutable.

SPRINT 2 · CERRAR LA VENTA Y ENTREGAR

7 Cesta y pago con Stripe

IMPRESCINDIBLE

> Montar la cesta del cliente y el pago con Stripe, que al confirmarse convierte la cesta en pedido y factura.

Es el apartado más grande que queda y el que el profesor pidió expresamente. `Cesta` y `LineaCesta` existen como entidades y no las usa nadie: no hay DAO, ni servicio, ni controlador. Hay que añadir la dependencia de Stripe (todavía no está en el pom), crear la sesión de pago, y recoger la confirmación en un webhook o en la vuelta del pago para marcar el pedido como PAGADO. Los datos de tarjeta los teclea el cliente en Stripe, nunca en un formulario nuestro.

Ojo con dos cosas que ya sabemos: una línea de cesta con cantidad 2 se expande en dos líneas de pedido, porque una línea de pedido es una clave; y la factura tiene que guardar copia del nombre y la dirección, o el día que un cliente se mude le cambiarán las facturas viejas.

Rama recomendada: `cesta-y-pago-stripe`

8 Claves de licencia, biblioteca y descarga

IMPRESCINDIBLE

> Acuñar la clave al pagar, enseñarla en la biblioteca del cliente y servir el ZIP solo a quien tenga esa clave.

`ClaveLicencia` es hoy una entidad huérfana: no hay DAO, ni servicio, ni una sola clave en la base. Es la pieza que cierra el circuito. Para un juego de colaborador la clave nace vendida al comprar y no cuelga de ninguna línea de compra; para un juego de distribuidor sale de las que entraron por una orden de compra. Falta también declarar la biblioteca en `Cliente` (el `@OneToMany` contra `ClaveLicencia`, el lado dueño ya está) y el controlador de descarga, que antes de servir el ZIP comprueba que la clave es de ese usuario y está VENDIDA. Acuérdate del índice ciego: `codigo_hash` lo rellena el servicio con `IndiceCiego.hash(valor)`, no se calcula solo.

Rama recomendada: `biblioteca-y-claves`

> **Cuidado con el orden.** Sin el apartado 9, los únicos juegos vendibles son los de colaborador y los de stock ilimitado, porque los de distribuidor no tienen claves. Si no da tiempo a hacer compras, la demo se monta con esos y se dice en la memoria.

9 Órdenes de compra y recepción de claves

IMPORTANTE

> Dar de alta órdenes de compra a un proveedor y recibir las claves que entran por cada línea.

Es la mitad izquierda del esquema: proveedor → compra → línea de compra → clave. `Compra` y `LineaCompra` existen como entidades, pero solo las menciona `DatosIniciales`: no hay CRUD. El CRUD de proveedores ya está hecho y sirve de punto de partida. Dos reglas que el modelo ya lleva dentro: cantidad nula significa ilimitada, y el empleado que firma la compra es obligatorio.

Rama recomendada: `compras-y-recepcion`

> Es lo primero que se sacrifica si los apartados 7 y 8 se comen el sprint: sin esto el ERP sigue enseñándose entero por el lado de la venta.

10 Notificaciones en tiempo real y tareas en segundo plano

NUEVO IMPRESCINDIBLE

> Avisar por WebSocket a quien tenga el panel abierto, y sacar de la petición el trabajo que no tiene por qué hacer esperar al usuario.

Es el **requisito 3.5** y el pom lo lleva desde el primer día: `spring-boot-starter-websocket` y su starter de prueba están declarados y **no hay ni una línea que los use**. Lo mismo con los hilos: la pantalla `admin/jobs` del prototipo enseña seis procesos con `@Async` y `@Scheduled`, y en el código no aparece ninguna de las dos anotaciones.

Los hilos. Una configuración con `@EnableAsync` y un `@Async` en el envío de correo: hoy `EmailServiceImpl` es síncrono, así que el gestor que activa a un cliente se queda esperando a que conteste el servidor de correo. Y un `@Scheduled` para la purga, que no es un adorno: en cuanto el apartado 8 reserve claves al iniciar el pago, una clave reservada cuyo pago no se termina queda bloqueada **para siempre** si nadie la libera. `ClaveLicencia.fechaReserva` existe justo para eso.

Los sockets. Un `WebSocketConfig` con `@EnableWebSocketMessageBroker`, un broker simple sobre `/topic` y el endpoint de conexión. Con dos avisos basta para justificar el requisito: el de **stock bajo**, cuando las claves disponibles de un juego caen por debajo de su `umbral_stock_bajo`, una columna que ya existe y que hoy no lee nadie, y el de **pedido nuevo** para el gestor de clientes. En el navegador, un script que se suscribe y pinta el aviso en la barra del panel.

Rama recomendada: `notificaciones-tiempo-real`

> El canal también tiene rol: un cliente no se puede suscribir a `/topic/inventario`. Las reglas de `SecurityConfig` son por URL y no cubren el socket, así que eso se configura aparte. El aviso de stock bajo depende del apartado 8; los hilos y la purga se pueden hacer antes.

11 Valoraciones de los juegos

NUEVO IMPORTANTE

> Que un cliente puntúe y comente un juego que tiene en su biblioteca, y que la nota media salga en la ficha y en la tarjeta del catálogo.

Es lo único de toda la lista que **necesita una migración nueva**, la V10: no hay entidad, ni tabla, ni columna, ni nada en el documento de diseño ni en el prototipo. Hace falta una entidad `Valoracion` y su tabla con `videojuego_id`, `cliente_id`, la puntuación de 1 a 5, un comentario opcional y la fecha, más un `UNIQUE (videojuego_id, cliente_id)`, que es lo que garantiza una sola valoración por cliente y juego sin comprobarlo a mano.

La regla que de verdad importa: solo valora quien tiene el juego, o sea quien tiene una clave VENDIDA suya. Si no, cualquiera puntúa cualquier cosa y la nota no significa nada. Eso lo comprueba el servicio contra `clave_licencia`, así que este apartado depende del 8.

La media **no se guarda**: se calcula con un `avg()` en el DAO al pintar la ficha o el catálogo. Desnormalizarla a una columna de `videojuego` solo tiene sentido si algún día pesa, y hoy no pesa. Hay que tocar `TiendaServiceImpl` para que la tarjeta lleve la nota, igual que ahora lleva la portada.

Rama recomendada: `valoraciones-de-juegos`

> Es alcance nuevo que no está en el documento de diseño: al hacerlo hay que añadirlo al PDF del apartado 14, a la figura de clases y a la del esquema de la base.

12 Arreglos sueltos del panel

IMPORTANTE

> Los detalles que quedan, los que no rompen nada pero un profesor encuentra en dos clics.

Al listado de clientes le falta el botón **Limpiar** que tienen los otros siete. La paginación de `ventas/pedidos` y `ventas/facturas` sigue siendo decorativa, con las flechas deshabilitadas y un 1 fijo, mientras los otros siete listados ya la tienen real. Y el filtro de facturas no tiene botón de enviar: solo funciona pulsando Enter.

Rama recomendada: `arreglos-panel`

13 Dejarlo listo para producción

IMPRESCINDIBLE

> Sacar el perfil dev de `application.properties`, pasar los secretos al entorno y desplegar la aplicación en algún sitio.

La aplicación tiene que acabar en producción porque los juegos del tercer proyecto se publicarán en ella. Hoy `spring.profiles.active=dev` está fijado en el fichero, así que un despliegue tal cual cargaría los usuarios de prueba con contraseña conocida, uno de ellos ADMIN. El perfil tiene que venir del entorno, igual que `LUDUSERP_AES_KEY`, y conviene separar esa clave de la del índice ciego, que ahora es la misma. Y hay que resolver dónde viven las subidas: `app.uploads.dir` apunta a una carpeta relativa al proyecto, que en un hosting que recrea el disco en cada despliegue se lleva por delante las imágenes y los ZIP.

Rama recomendada: `preparar-produccion`

14 Documentación de entrega: PDF, prototipo y memoria

IMPRESCINDIBLE

> Rehacer el documento de diseño para que cuente lo que de verdad se ha construido, y volver a sincronizar prototipo y memoria.

El PDF va con más de veinte correcciones pendientes apuntadas en la memoria: sobra la clase Desenvolupador y las tablas de permisos, falta el actor COLABORADOR entero, la jerarquía de Usuari abstracta, los siete roles en vez de cinco, Stripe en lugar del pago simulado, y Categoria y Plataforma como clases. Además promete cosas que no se usan (MapStruct, Bootstrap, nombres en inglés) y dice Spring Boot 3 cuando el pom es 4.1.1. A esa lista se suman las **valoraciones** del apartado 11 y las **notificaciones** del 10.

La memoria (`docs/memoriaClaudeByAleix.txt`) se quedó en el 4 de noviembre y no recoge ventas, correo, paginación, el CRUD de videojuegos, las imágenes, las fotos de perfil ni la tienda. Es trabajo de varias horas y no se puede dejar para el último día, porque es la mitad de lo que se corrige.

Rama recomendada: `documentacion-entrega`

15 El resto de las pruebas y las chapuzas conocidas

SI SOBRA TIEMPO

> Cuatro pruebas de servicio sobre el test de contexto del apartado 5, y barrer la deuda pequeña apuntada desde octubre.

Ningún servicio lleva `@Transactional` salvo los de clientes y departamentos, así que validar y guardar no es atómico: dos altas simultáneas con el mismo NIF pasan las dos la comprobación y la segunda muere con un 500 en vez del mensaje. No hay `mvnw`. El healthcheck de MySQL de `compose.yaml` debería ser CMD-SHELL. `nbactions.xml` arrastra `-noverify`, obsoleta desde JDK 13. Y el pom sigue con artifactId `demo` y groupId `com.example`.

Rama recomendada: `tests-y-deuda`

#	APARTADO	RAMA	SPRINT	NECESIDAD
1	Punto de control: recrear la base, arrancar y mergear	`punto-control-esquema`	1, día 1	Imprescindible
2	Cerrar las decisiones abiertas	`decisiones-pendientes`	1, día 1	Imprescindible
3	Arreglar las dos pantallas rotas	`arreglos-rotos`	1	Imprescindible
4	Datos de prueba de pedidos y facturas	`datos-prueba-ventas`	1	Importante
5	Un test que arranque el contexto	`test-contexto`	1	Importante
6	El colaborador sube sus juegos	`juegos-colaborador`	1	Imprescindible
7	Cesta y pago con Stripe	`cesta-y-pago-stripe`	2	Imprescindible
8	Claves, biblioteca y descarga	`biblioteca-y-claves`	2	Imprescindible
9	Órdenes de compra y recepción	`compras-y-recepcion`	2	Importante
10	Notificaciones en tiempo real y tareas en segundo plano	`notificaciones-tiempo-real`	2	Imprescindible
11	Valoraciones de los juegos	`valoraciones-de-juegos`	2	Importante
12	Arreglos sueltos del panel	`arreglos-panel`	2	Importante
13	Dejarlo listo para producción	`preparar-produccion`	2	Imprescindible
14	Documentación de entrega	`documentacion-entrega`	1 y 2	Imprescindible
15	El resto de pruebas y deuda pequeña	`tests-y-deuda`	2	Si sobra tiempo

> Si el tiempo se tuerce, el orden de recorte es: primero el 15, después el 11, luego el 9 y por último el 12. Lo que no se puede recortar es la cadena 6 → 7 → 8, ni el 10, que es requisito de la asignatura, ni el 13 ni el 14: sin ellos no hay ni aplicación en producción ni documento que entregar.